Visitor

Season: 3 · Behavioral · Category: GoF · Level: Intermediate

Intent

- Add operations to object structures without changing classes.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Airport security scanning different bag types.

When to Use

- See Intent.
- The design force named in Intent shows up more than once.

When Not to Use

- Overkill for one-off behavior.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor;

sealed interface Element permits CircleEl, RectEl {
    <R> R accept(Visitor<R> visitor);
}

record CircleEl(double r) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

record RectEl(double w, double h) implements Element {
    public <R> R accept(Visitor<R> visitor) { return visitor.visit(this); }
}

interface Visitor<R> {
    R visit(CircleEl c);
    R visit(RectEl r);
}

/** Visitor vs Java 25 pattern matching on sealed types. */
public final class ShapeVisitor {
    private ShapeVisitor() {}

    public static double area(Element e) {
        return switch (e) {
            case CircleEl(var r) -> Math.PI * r * r;
            case RectEl(var w, var h) -> w * h;
        };
    }
```

Classic Implementation (2)

```
    public static double areaViaVisitor(Element e) {
        return e.accept(new Visitor<>() {
            public Double visit(CircleEl c) { return Math.PI * c.r() * c.r(); }
            public Double visit(RectEl r) { return r.w() * r.h(); }
        });
    }
}
```

Modern Java 25 / Functional

- Java 25 pattern matching often replaces Visitor/Interpreter boilerplate.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/behavioral/visitor/
```

Pitfalls

- Prefer sealed + switch over classic Visitor in Java 25.
- Strategy as a function value.
- Observer: unsubscribe to avoid leaks.

Interview Q&A

- Q: Key interview angle for Visitor? A: Name the force (undo, open/closed, open for extension) and a modern alternative.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Add operations to object structures without changing classes.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Airport security scanning different bag types. [0:40] Intent — State the design force in one sentenc...